Парадигми паралельного програмування

Природний паралелізм задач

- Довгий час комп'ютери і комп'ютерні технології асоціювались з поняттям алгоритму і *алгоритмізацією* обчислювальних процесів.
- Проте алгоритм – це запис *послідовності* обчислень, незважаючи на те, що багато обчислювальних (і необчислювальних) задач передбачають для свого вирішення так званий *природний паралелізм* виконання операцій, тобто такий паралелізм виконання операцій, що *допускається специфікацією* самої задачі, а не випливає з *алгоритму її вирішення*.

Простий приклад

- Наприклад, для обчислення суми елементів числового вектора $S = a_1+a_2+\ldots + a_n$, при $n>2$, зовсім не обов'язково *послідовно перебирати* елементи вектора, накопичуючи їх суму, як це передбачають засоби більшості (послідовних) мов програмування.
- Такий алгоритм вимагає за порядком $O(n)$ кроків, тоді як прийом логарифмічного парування, при якому сумування незалежних пар елементів вектора відбувається паралельно, має для оцінки тієї ж часової складності величину порядку $O(\log_2 n)$.

Алгоритмізація vs паралелизація

- Таким чином, алгоритмізація в її традиційному розумінні, як *упослідовнення* обчислювальних процесів для їх виконання на комп'ютерах, для паралельних обчислювальних систем є, принаймні, *передчасним, а то й шкідливим*, етапом підготовки задач.
- Виявлення природного паралелізму при виконанні операцій ще *на етапі аналізу предметної області* і отримання специфікації задачі стає *першою* необхідною умовою ефективності обчислень.
- Ця умова задовольняється шляхом *декомпозиції* задачі на підзадачі, що є незалежними, і можуть виконуватись паралельно.
- *Другою* такою умовою має бути умова *ефективного відображення* застосованої декомпозиції на паралельну архітектуру.
- І якщо це відбувається, то *третьою* умовою має бути відповідна *система паралельного програмування*, що здійснює реалізацію вибраної декомпозиції на архітектурі мультипроцесорної системи.

Про парадигми

- *Парадигмою* прийнято називати сукупність методів та засобів для виконання певної діяльності, пов'язаних усталеною схемою їх застосування у цій діяльності (у перекладі з грецької – приклад, взірець).
- Звичайно, парадигми обмежуються певними умовами або класами застосувань і *не є універсальними*.
- Нижче розглядаються декілька парадигм паралельного програмування для машин типу MIMD.
- Логічний спосіб одержання цих парадигм можна уявити як перетворення відомої формули швейцарського математика Н.Вірта :

Програми = Алгоритми + Структури даних

- шляхом застосування різних підходів до розподілу (декомпозиції) даних і програмного коду у просторі і часі.

Паралелізм за даними

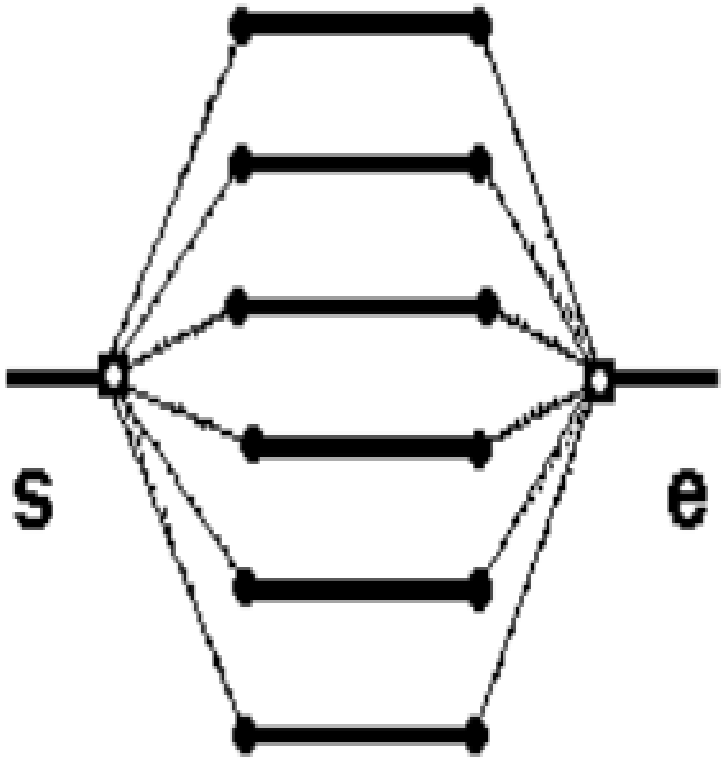
- За цією парадигмою наголос робиться на *декомпозицію структур даних*, і формула Вірта для цього випадку приймає вигляд:

Паралельні програми = Алгоритми + Розподілені структури даних

- Інша назва для цієї парадигми – геометричний паралелізм.
- Ця парадигма передбачає *розподіл області даних* на (геометричні) частини між паралельними гілками обчислень таким чином, що програми, які обробляють ці частини, є *незалежними* і можуть виконуватись одночасно (паралельно).

Схема обчислень – паралельно-послідовна композиція

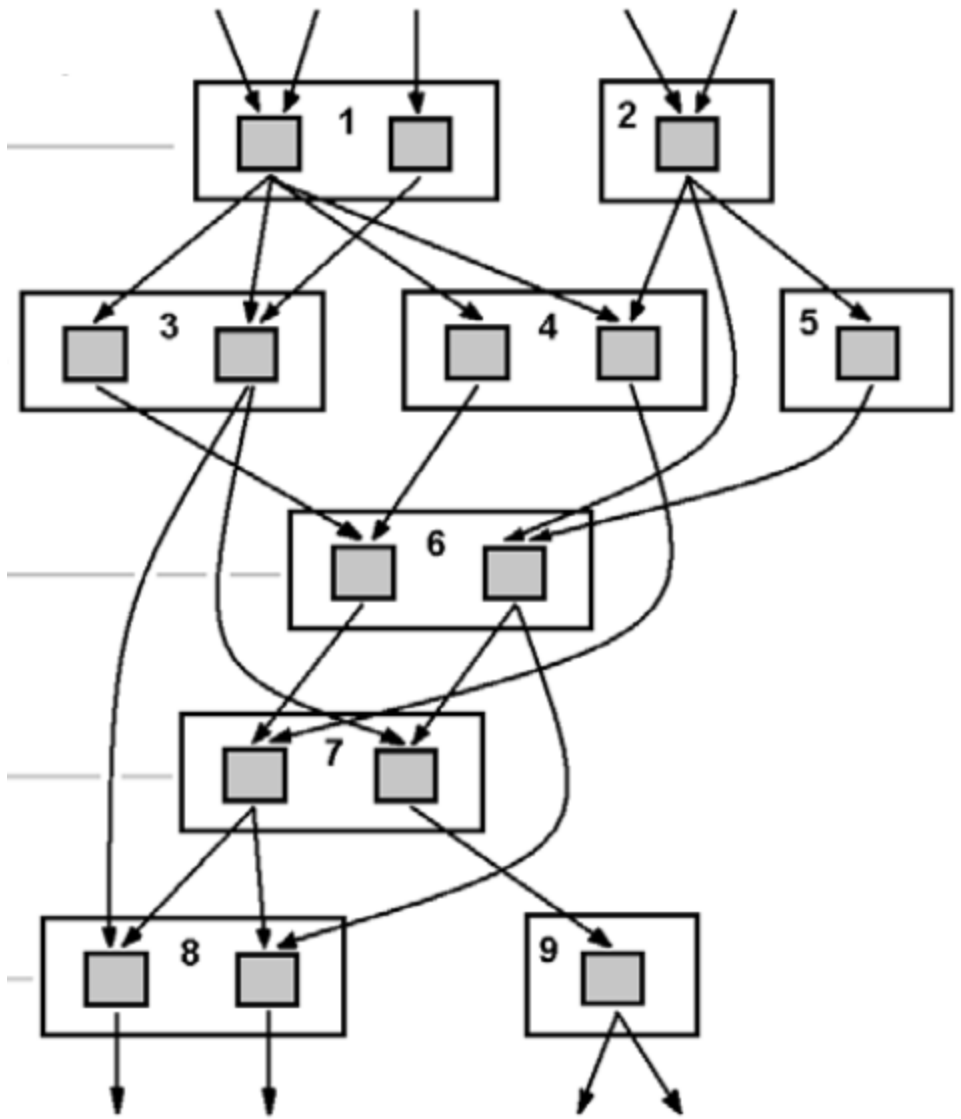
- Схема обчислень такої парадигми – *паралельно-послідовна композиція* сегментів обчислень, де паралельні ділянки означають, як правило, виконання однієї і тієї ж програми, але над різним областями даних .
- Поширені приклади застосування такої парадигми – *задачі лінійної алгебри*, сіткові методи обчислювальної математики.
- Паралелізм за даними характерний своїм *регулярним (однорідним)* характером паралельних ділянок обчислень, і тому досить легко реалізується на машинах типу SIMD або MIMD.
- Існує також загальна, об'єднавча, парадигма *SPMD* (Single Program Multiple Data) для обох типів архітектур.



Паралелізм за керуванням

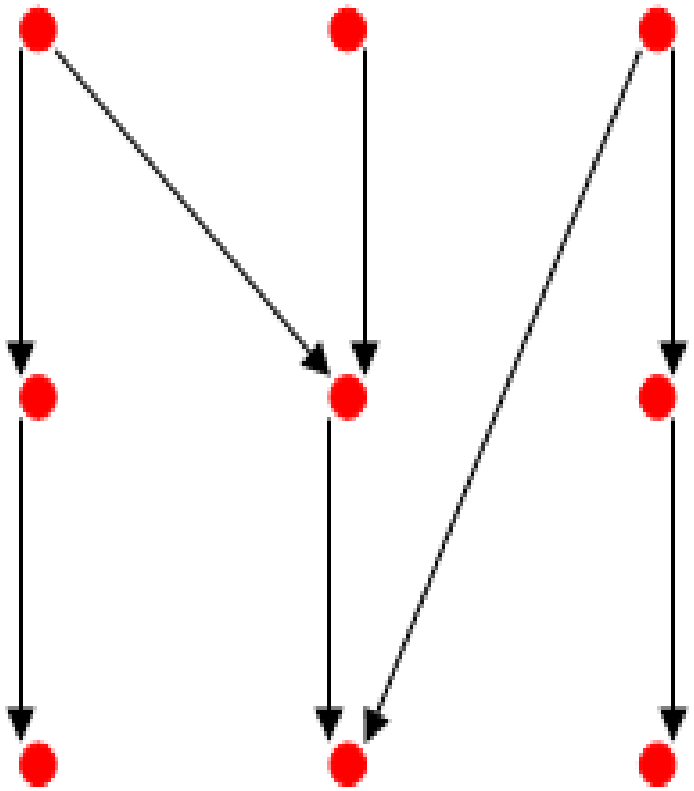
- Інша назва для цієї парадигми – *алгоритмічний паралелізм*.
- Акцентує увагу на *декомпозиції алгоритмів* на окремі підзадачі, яка дозволяють паралельне їх виконання.
- Ця парадигма є більш загальною і широкою доля використання, бо допускає ефективне розпаралелювання і для *нерегулярних схем* обчислень задач.
- Схема обчислень такої парадигми – граф підзадач. Вхідні стрілки для початкових вершин означають запуск відповідних початкових підзадач і отримання початкових даних, а решта стрілок – передачу керування.
- Для цієї парадигми формула Вірта має вигляд:

Паралельні програми = Паралельні алгоритми + Структури даних

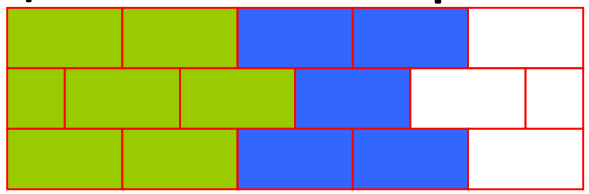
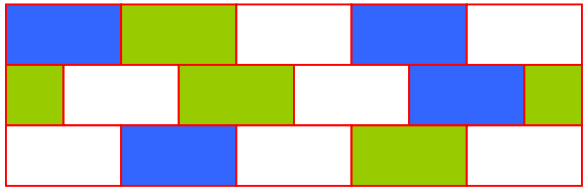
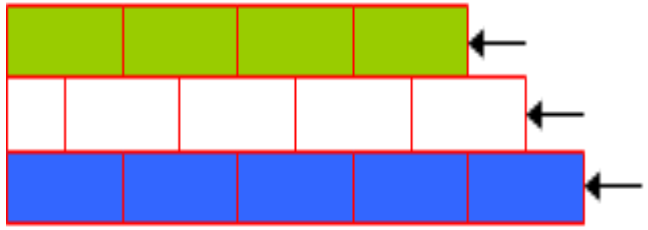


Потоковий паралелізм

- Це така схема взаємодії паралельних процесів, при якому зв'язок між ними за керуванням і за даними поєднано в один, який і називається *потоковим.*
- Це означає, що кожний процес активується тільки тоді, коли *готові дані* для нього. Потокові схеми можуть зображатись, в загальному випадку, довільними графами.
- Проте для складних потокових схем важко передбачити високі показники продуктивності.
- Тому найбільш поширеними є випадки потокових схем, наприклад, лінійних, що являють собою *сукупність конвеєрів*, утворених паралельними процесорами з ациклічними потоковими зв'язками (див. рис.).



Приклад – стіна Дж. Фокса

- Класи застосувань різних парадигм паралельного програмування *не мають чітко окреслених границь*. Одну і ту ж задачу можна подати у різних парадигмах, де кожна матиме свої переваги і недоліки.
- “Будівельний“ приклад паралельних обчислень.
-  паралелізм за даними
-  паралелізм за керуванням
-  потоковий паралелізм

Завдання на самостійну роботу

- Подати задачу сумування елементів числового вектора $\boldsymbol{s} = \sum_{1}^{n} \boldsymbol{a_i}$ у трьох парадигмах паралелізму:
  - за даними;
  - за керуванням;
  - у потоковій парадигмі